

JunubLink AI - junublink.py

Note: Arabic text and emoji appear as [Arabic] and [emoji] in this PDF. The real junublink.py file contains them. Upload the .py file for the working code.

```
"""
```

JunubLink AI - starter prototype (WhatsApp-style assistant for students and hustlers in Juba).

Run in the terminal to test: `python junublink.py`

Run as a WhatsApp webhook: `pip install flask twilio anthropic`

`python junublink.py --serve`

Set ANTHROPIC_API_KEY to turn on the "explain" feature (module 7).

IMPORTANT: Arabic wording below is standard-ish Arabic. Have Juba Arabic speakers review and fix it before real users see it.

```
"""
```

```
import json
import os
import re
import statistics
import sys
import time
from urllib.parse import urlparse
```

```
DATA_DIR = os.path.dirname(os.path.abspath(__file__))
PRICES_FILE = os.path.join(DATA_DIR, "prices.json")
MODEL = os.environ.get("JUNUBLINK_MODEL", "claude-sonnet-5-5")
```

```
# ----- data
```

```
# Only add listings you have verified yourself. Keep deadlines current.
```

```
OPPORTUNITIES = {
    "jobs": [], # add verified jobs: {"title":..., "deadline":..., "how":...}
    "scholarships": [], # add verified scholarships
    "workshops": [
        {"title": "Commonwealth Youth Council - Wellbeing Champions (Are You Okay?)",
         "deadline": "4 Oct 2026", "how": "cycofficial.org/are-you-okay"},
        {"title": "timbuktoo UniPods AI Innovation Programme, Cohort 2",
         "deadline": "30 Oct 2026", "how": "Apply via timbuktoo's official channels"},
    ],
}
```

```
LOCATIONS = {} # e.g. {"konyokonyo market": "Near ... , Juba. Open ..."}

# Domains you have verified as official. Add more only after checking them.
OFFICIAL_DOMAINS = {"cycofficial.org", "thecommonwealth.org", "undp.org", "un.org"}
SHORTENERS = {"tinyurl.com", "bit.ly", "t.co", "cutt.ly", "is.gd", "rb.gy"}
RED_FLAGS = [
    (r"\b(processing|application|registration|admin)\s+fee\b", "asks for a fee"),
    (r"\b(send|pay|transfer)\b.{0,30}\b(money|fee|money|mobile money)\b", "asks for money"),
    (r"\b(passport|bank account|pin|password|otp)\b", "asks for private documents or codes"),
    (r"\bguaranteed\s+(visa|job|scholarship|admission)\b", "promises a guaranteed result"),
    (r"\b(urgent|last chance|act now|within 24 hours)\b", "pressures you to hurry"),
    (r"@(\bgmail|yahoo|hotmail)\.com\b", "uses a free email for an 'official' offer"),
]
```

```
TEXT = {
    "menu": {
        "en": ("Welcome to JunubLink AI \U0001F44B Reply with a number:\n"
              "1 Jobs\n2 Scholarships & grants\n3 Workshops & programmes\n"
              "4 Market prices\n5 Locations\n6 Is this offer real?\n"
              "7 Explain a topic\n\nType 'ar' for Arabic, 'en' for English."),
        "ar": ("[Arabic] [Arabic] [Arabic] [Arabic] [Arabic] \U0001F44B [Arabic] [Arabic]:\n"
              "1 [Arabic]\n2 [Arabic] [Arabic]\n3 [Arabic] [Arabic]\n"
              "4 [Arabic] [Arabic]\n5 [Arabic]\n6 [Arabic] [Arabic] [Arabic] [Arabic]\n"
              "7 [Arabic] [Arabic] [Arabic]\n\n[Arabic] 'en' [Arabic]."),
    },
    "none": {"en": "No verified listings yet. Check back soon.",
             "ar": "[Arabic] [Arabic] [Arabic] [Arabic] [Arabic]. [Arabic] [Arabic]."},
    "ask_offer": {"en": "Paste the offer text or link here and I will check it.",
                  "ar": "[Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic]."},
    "ask_topic": {"en": "Type the topic or paste the text you want explained.",
                  "ar": "[Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic]."},
    "ask_loc": {"en": "Type the place name.", "ar": "[Arabic] [Arabic] [Arabic]."},
    "price_help": {"en": ("To share a price: price <item> <amount> <market>\n"
                          "Example: price tomatoes 3000 konyokonyo\n")
}
```

```

        "To see a price: price <item>"),
        "ar": ("[Arabic] [Arabic] [Arabic]: price <[Arabic]> <[Arabic]> <[Arabic]>\n"
               "[Arabic] [Arabic] [Arabic]: price <[Arabic]>")),
    }

sessions = {} # user_id -> {"lang": "en|"ar", "mode": None|"offer"|"topic"|"loc"}

def t(key, lang):
    return TEXT[key][lang]

# ----- prices
def _load_prices():
    if os.path.exists(PRICES_FILE):
        with open(PRICES_FILE, encoding="utf-8") as f:
            return json.load(f)
    return []

def _save_prices(rows):
    with open(PRICES_FILE, "w", encoding="utf-8") as f:
        json.dump(rows, f, ensure_ascii=False, indent=1)

def price_command(text, lang):
    parts = text.split()
    rows = _load_prices()
    if len(parts) >= 4 and parts[2].replace(",", "").isdigit():
        rows.append({"item": parts[1].lower(), "amount": int(parts[2].replace(",", "")),
                     "market": " ".join(parts[3:]).lower(), "ts": time.time()})
        _save_prices(rows)
        return "Thanks! Price saved." if lang == "en" else "[Arabic]! [Arabic] [Arabic]."
    if len(parts) == 2:
        week = time.time() - 7 * 86400
        found = [r["amount"] for r in rows if r["item"] == parts[1].lower() and r["ts"] > week]
        if not found:
            return ("No recent reports for that item." if lang == "en"
                    else "[Arabic] [Arabic] [Arabic] [Arabic] [Arabic].")
        med = int(statistics.median(found))
        return (f"{parts[1]}: about {med} (from {len(found)} reports this week)."
                if lang == "en" else f"{parts[1]}: [Arabic] {med} ([Arabic] {len(found)}) [Arabic] [Arabic] [Arabic] [Arabic].")
    return t("price_help", lang)

# ----- scam check
def check_offer(text, lang):
    flags, good = [], []
    for url in re.findall(r"https?://\S+|\b[\w-]+\.(?:com|org|net|co|info)\S*", text):
        host = urlparse(url if "/" in url else "/" + url).netloc.lower().removeprefix("www.")
        if host in SHORTENERS:
            flags.append(f"shortened link ({host}) hides the real address")
        elif any(host == d or host.endswith("." + d) for d in OFFICIAL_DOMAINS):
            good.append(f"link goes to {host}, which is on our verified list")
    for pattern, why in RED_FLAGS:
        if re.search(pattern, text, re.I):
            flags.append(why)
    if flags:
        head = "WARNING - red flags found:" if lang == "en" else "[Arabic] - [Arabic] [Arabic] [Arabic]:"
    elif good:
        head = ("No red flags, and the link looks official. Still confirm on the organization's own website:"
                if lang == "en" else "[Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic]. [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic]:")
    else:
        head = ("Could not confirm this offer. Verify it on the organization's official website before you act:"
                if lang == "en" else "[Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic] [Arabic]:")
    lines = [head] + ["- " + x for x in flags + good]
    lines.append("Never pay to apply." if lang == "en" else "[Arabic] [Arabic] [Arabic] [Arabic] [Arabic].")

```

```

return "\n".join(lines)

# ----- explain
def explain(topic, lang):
    key = os.environ.get("ANTHROPIC_API_KEY")
    if not key:
        return "Explain feature is off (no API key)."
    import anthropic
    target = ("simple Arabic as spoken in Juba, South Sudan (Juba Arabic), using short sentences"
              if lang == "ar" else "very simple English")
    prompt = (f"Explain this to a student in {target}. Use one everyday example from "
              f"South Sudan. Keep it under 120 words. If you are unsure of a term, say "
              f"so.\n\n{topic}")
    msg = anthropic.Anthropic(api_key=key).messages.create(
        model=MODEL, max_tokens=500, messages=[{"role": "user", "content": prompt}])
    return msg.content[0].text

# ----- main logic
def list_items(category, lang):
    items = OPPORTUNITIES.get(category, [])
    if category == "scholarships":
        items = items + OPPORTUNITIES.get("grants", [])
    if not items:
        return t("none", lang)
    return "\n\n".join(f"{i['title']}\nDeadline: {i['deadline']}\nHow: {i['how']}" for i in items)

def handle_message(user_id, text):
    s = sessions.setdefault(user_id, {"lang": "en", "mode": None})
    text = (text or "").strip()
    low = text.lower()
    lang = s["lang"]

    if low in ("ar", "en"):
        s["lang"], s["mode"] = low, None
        return t("menu", low)
    if low.startswith("price"):
        s["mode"] = None
        return price_command(text, lang)
    if s["mode"] == "offer":
        s["mode"] = None
        return check_offer(text, lang)
    if s["mode"] == "topic":
        s["mode"] = None
        return explain(text, lang)
    if s["mode"] == "loc":
        s["mode"] = None
        return LOCATIONS.get(low, t("none", lang))

    routes = {"1": lambda: list_items("jobs", lang),
              "2": lambda: list_items("scholarships", lang),
              "3": lambda: list_items("workshops", lang),
              "4": lambda: t("price_help", lang)}
    if low in routes:
        return routes[low]()
    modes = {"5": ("loc", "ask_loc"), "6": ("offer", "ask_offer"), "7": ("topic", "ask_topic")}
    if low in modes:
        s["mode"] = modes[low][0]
        return t(modes[low][1], lang)
    return t("menu", lang)

# ----- entry points
def serve():
    from flask import Flask, request
    from twilio.twiml.messaging_response import MessagingResponse
    app = Flask(__name__)

    @app.post("/whatsapp")
    def whatsapp():
        reply = handle_message(request.form.get("From", "unknown"), request.form.get("Body", ""))
        resp = MessagingResponse()

```

```
        resp.message(reply)
        return str(resp)

app.run(port=5000)

if __name__ == "__main__":
    if "--serve" in sys.argv:
        serve()
    else:
        print("JunubLink AI test chat. Ctrl+C to quit.\n")
        print(handle_message("me", "hi"))
        while True:
            print("\n" + handle_message("me", input("> ")))
```